38、四种启动方式

全志V3S嵌入式驱动开发（四种启动方式）

原创

嵌入式-老费 已于 2023-08-29 15:28:28 修改

阅读量1.7k 收藏 2

点赞数 1

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

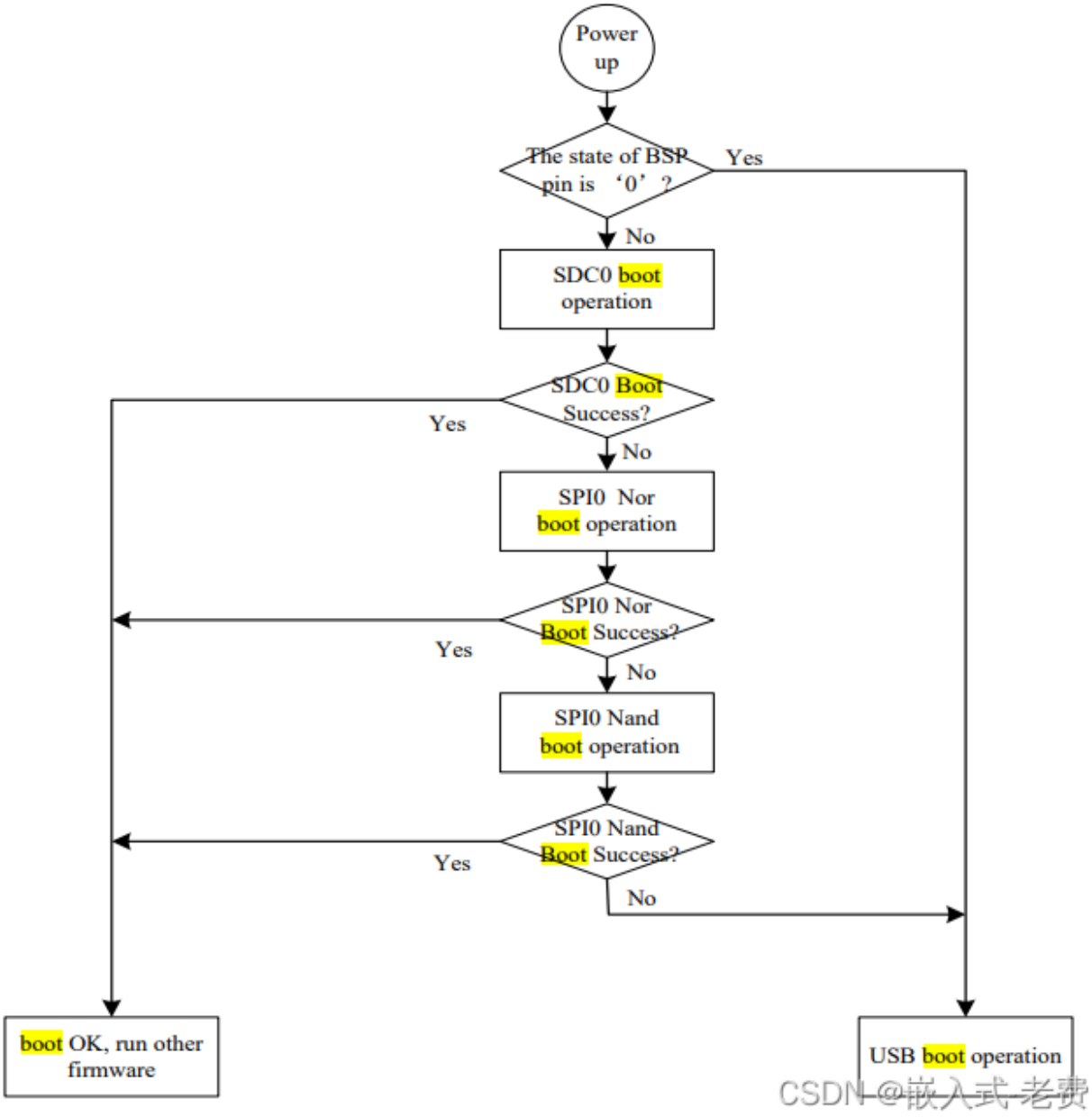
](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 该博客详细分析了全志V3S芯片的四种启动方式，包括uboot加载、uboot后的处理、dtb文件处理、kernel以及rootfs。着重讨论了不同启动模式下的uboot处理方式，以及dtb文件在micro SD卡、SPI NOR和SPI NAND之间的区别。同时，提到了kernel和rootfs的处理策略，以及如何优化存储空间利用。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

对于v3s的启动方式分析，怎么研究都不为过。对我们整个系列文章比较熟悉的同学来说，这幅图真的是太熟悉了，



整个流程不复杂。它主要是告诉我们，v3s加载的顺序是怎么样的。首先，看BSP的pin有没有短接，如果短接了，就直接跳转到USB boot operation。这个短接pin其实就

是spi接口上的短接pin。接着依次看sd0、spi nor、spi nand的启动顺序。注意看，如果上面三个还是不成功，也同样会进入到USB boot operation。 **进入USB的状态之后，我们就可以用sunxi-fel或者xfel命令进行操作了。**

1、uboot加载

统一来看，四种启动模式的uboot处理方式是不一样的。对USB来说，可以通过下面这条命令，直接加载uboot程序，

```
sudo ../sunxi-tools-3s-spi/sunxi-fel uboot u-boot-sunxi-with-spl.bin
```

而如果加载方式是micro sd卡，则依赖ubuntu的dd命令来解决。与此相比较，spi-nor需要先通过sunxi-fel写入uboot内容，然后才能从spi-nor加载成功。spi-nand也是一样，不过它需要的命令是xfel，当然也是需要先写入uboot，才能从spi-nand正确加载的。

uboot是linux系统中非常重要的内容，不同启动方式uboot是不能一样的。 对于v3s来说，它还有一个很重要的地方。那就是，它的uboot文件，其实是两个uboot合并起来的，这个可以通过打印看的出来，

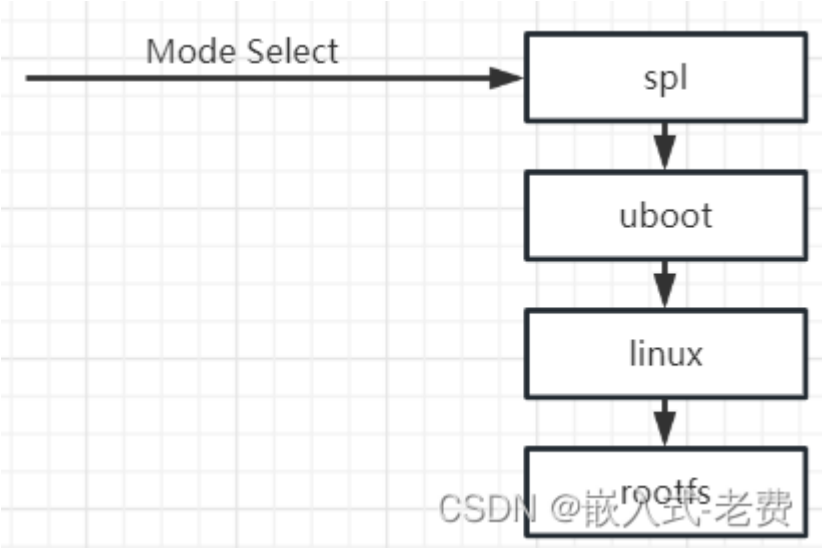
```
U-Boot SPL 2022.04 (Jun 26 2023 - 22:39:31 +0800)
DRAM: 64 MiB
Trying to boot from sunxi SPI-NAND
sunxi SPI-NAND: Found Macronix MX35LF1GE4AB (c212c2)
sunxi SPI-NAND: Loading u-boot from 0x20000
sunxi SPI-NAND: u-boot hcrc OK!


U-Boot 2022.04 (Jun 26 2023 - 22:39:31 +0800) Allwinner Technology

CPU:   Allwinner V3s (SUN8I 1681)
Model: Lichee Pi Zero
DRAM:  64 MiB
Core:  23 devices, 11 uclasses, devicetree: separate
WDT:   Not starting watchdog@1c20ca0
MMC:   mmc@1c0f000: 0
Loading Environment from FAT... Card did not respond to voltage select!
: -110
In:    serial@1c28000
Out:   serial@1c28000
Err:   serial@1c28000
```

```
Net:   No ethernet found.
Hit any key to stop autoboot:  0
Reading 131072 byte(s) (64 page(s)) at offset 0x00100000
Reading 5242880 byte(s) (2560 page(s)) at offset 0x00120000
Kernel image @ 0x41000000 [ 0x000000 - 0x48e908 ]
## Flattened Device Tree blob at 41800000
   Booting using the fdt blob at 0x41800000
   Loading Device Tree to 42df9000, end 42dff1e7 ... OK
```

前面6行是第一个uboot的打印，剩下来的内容是第二个uboot的打印。因此，整个加载的过程严格意义上说是这样的，



2、uboot加载以后的处理

对于几乎所有的soc来说，只要能加载uboot，后续的操作都是类似的，即加载kernel、dtb和rootfs。 **只是对v3s来说，USB阶段的主要目的就是加载和烧入uboot。** 剩下来的主要内容就是继续引导kernel、dtb和rootfs。这个时候，就可以不考虑USB的引导方式了。我们会重点分析micro sd卡、spi-nor、spi-nand三种方式。 **进入uboot之后，一般需要烧入kernel、dtb和rootfs，非常建议大家用uboot自带的ymodem命令来解决，非常方便，只是速度慢一点。** micro sd卡一般还是通过ubuntu的dd命令烧入，不受影响。

3、dtb文件的处理

目前从dtb的角度来说，micro sd卡和spi-nor可以归为一类，而spi-nand归为一类的。两者的区别主要就是arch/arm/boot/dts/sun8i-v3s-licheepi-zero.dts，区别有两个，

第一就是是否注释bootargs，

```
/*bootargs = "console=ttyS0,115200 earlyprintk panic=5 rootwait
mtdparts=spi0.0:1M(uboot)ro,128k(dtb)ro,5M(kernel)ro,-(rootfs)
ubi.mtd=3 root=ubi0:rootfs rw rootfstype=ubifs";*/
```

第二就是spi的驱动提示，对于spi-nor来说，它的驱动是这样的，

```
&spi0 {
        status ="okay";
        mx25l25645g:mx25l25645g@0 {
                compatible = "jedec,spi-nor";
                reg = <0x0>;
                spi-max-frequency = <50000000>;
                #address-cells = <1>;
                #size-cells = <1>;
        };
};
```

而对于spi-nand来说，驱动是这样的，

```
&spi0 {
    status ="okay";

    spi_nand: spi_nand@0 {
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "spi-nand";
        reg = <0>;
        spi-max-frequency = <50000000>;

        partition@0 {
            label = "uboot";
            reg = <0x0 0x100000>;
            read-only;
        };

        partition@100000 {
            label = "dtb";
            reg = <0x100000 0x20000>;
            read-only;
        };
```

```
        partition@120000 {
            label = "kernel";
            reg = <0x120000 0x500000>;
            read-only;
        };

        partition@620000 {
            label = "rootfs";
            reg = <0x620000 0x79E0000>;
        };
    };
};
```

对于micro sd卡启动来说，为了方便，我们一般默认选用和spi-nor一样的dtb文件就可以了。当然，如果micro sd卡启动后，发现板子上有一块spi-nand，其实dtb也可以修改成这样的，

```
&spi0 {
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_pins>;
        status = "okay";

        flash@0 {
                #address-cells = <1>;
                #size-cells = <1>;
                compatible = "spi-nand";
                reg = <0>;
                spi-max-frequency = <40000000>;

        partitions {
                compatible = "fixed-partitions";
                #address-cells = <1>;
                #size-cells = <1>;

            partition@0 {
                label = "all";
                reg = <0x000000 0x8000000>;
            };
        };
```

```
        };
};
```

这就是dtb的最大区别了。

4、kernel

不管哪种启动方式，kernel都是一样的，也就是说三者可以使用同一份zImage，这是很方便的。 使用过程中有两个地方需要注意下，第一，就是注意控制zImage的大小，一些不需要的功能可以不编译；第二，对于比较大的驱动模块，可以从kernel脱离，放到文件系统当中，这样zImage就不会一直发生改变，调试也比较方便。

5、rootfs

文件系统和kernel一样，最好只烧入一次，这样比较省心，不容易出错。kernel是通过动态加载ko的方式来保持zImage的稳定。而rootfs，则可以通过vsftpd、telnet这样的软件，实现相关程序、库、配置文件的修改，这样也是比较方便的。

此外，spi-nor一般较小，spi-nand和sd卡则空间富裕。 **这个时候，可以把相关程序、文件拷贝到u盘上面执行，等到最后发布的时候再灵活决定，这样是比较合适的一种做法。**